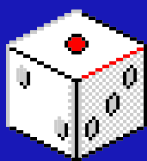
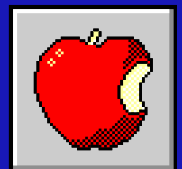


ROVER CHALLENGE

WITH AWS JUPYTER'S
NOTEBOOK



Presented By: Gavi, George, & Jocelyn



ABSTRACT



Our focus: Build an algorithm to quickly navigate a rover along a complex path to the location of an injured firefighter, who needs assistance. The data package includes high-resolution NAIP imagery, and an accurate road network for the same area.



Methodology and Results Found

Step 1:

- Plot out Guidance, Navigation, and Control System Algorithm as a pseudo-code for the robot's trajectory and method of movement

Step 2:

- Converted relevant tif files to manipulate arrays to represent obstacles and traversable data points for a path

Step 3:

- Use rasterio package functions to derive arrays of correct dimensions from imagery data
 - 0 = traversable
 - 1 = obstacle
 - 2 = path

Step 4:

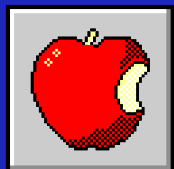
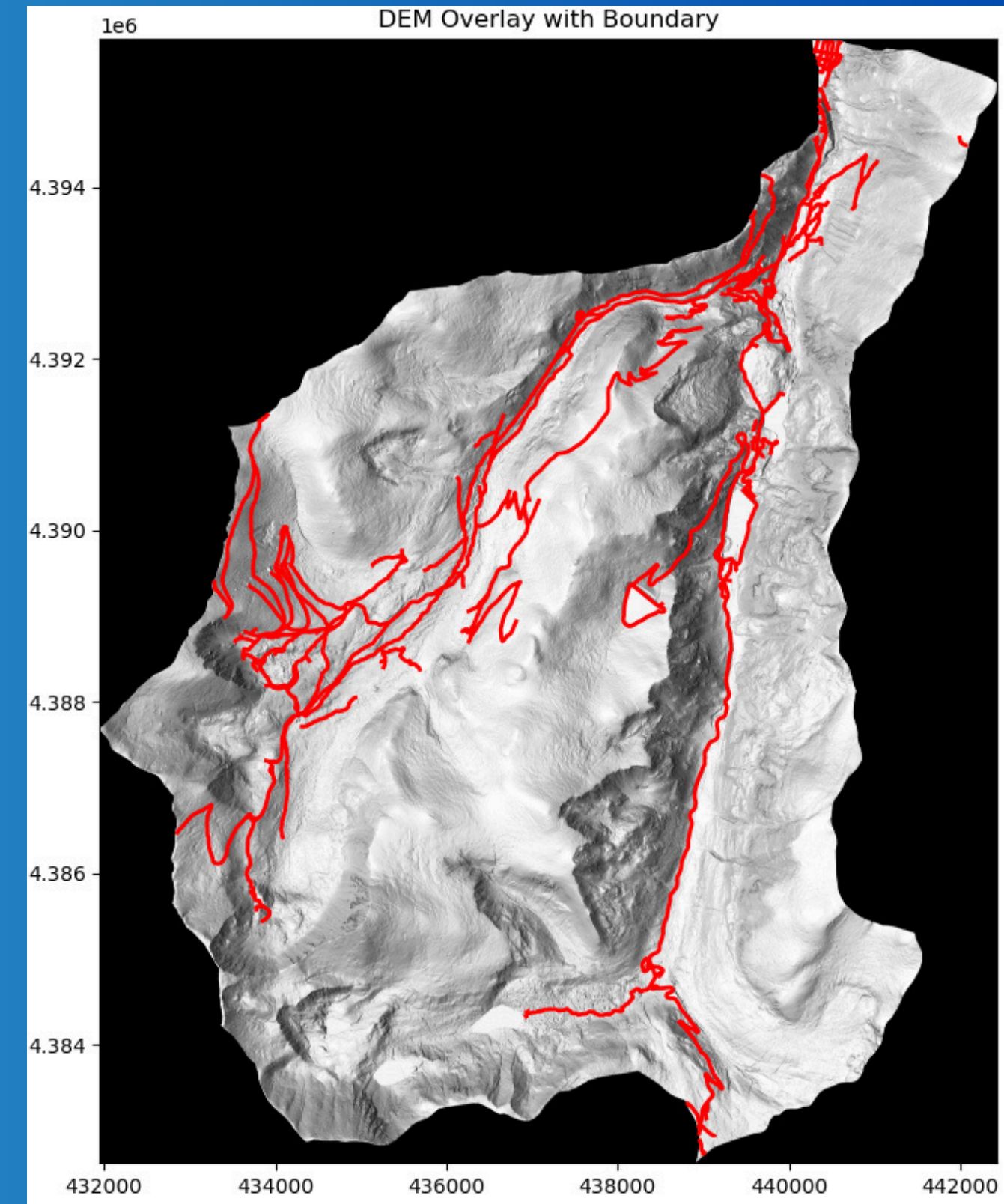
- Extract each band within the data into 2-D array

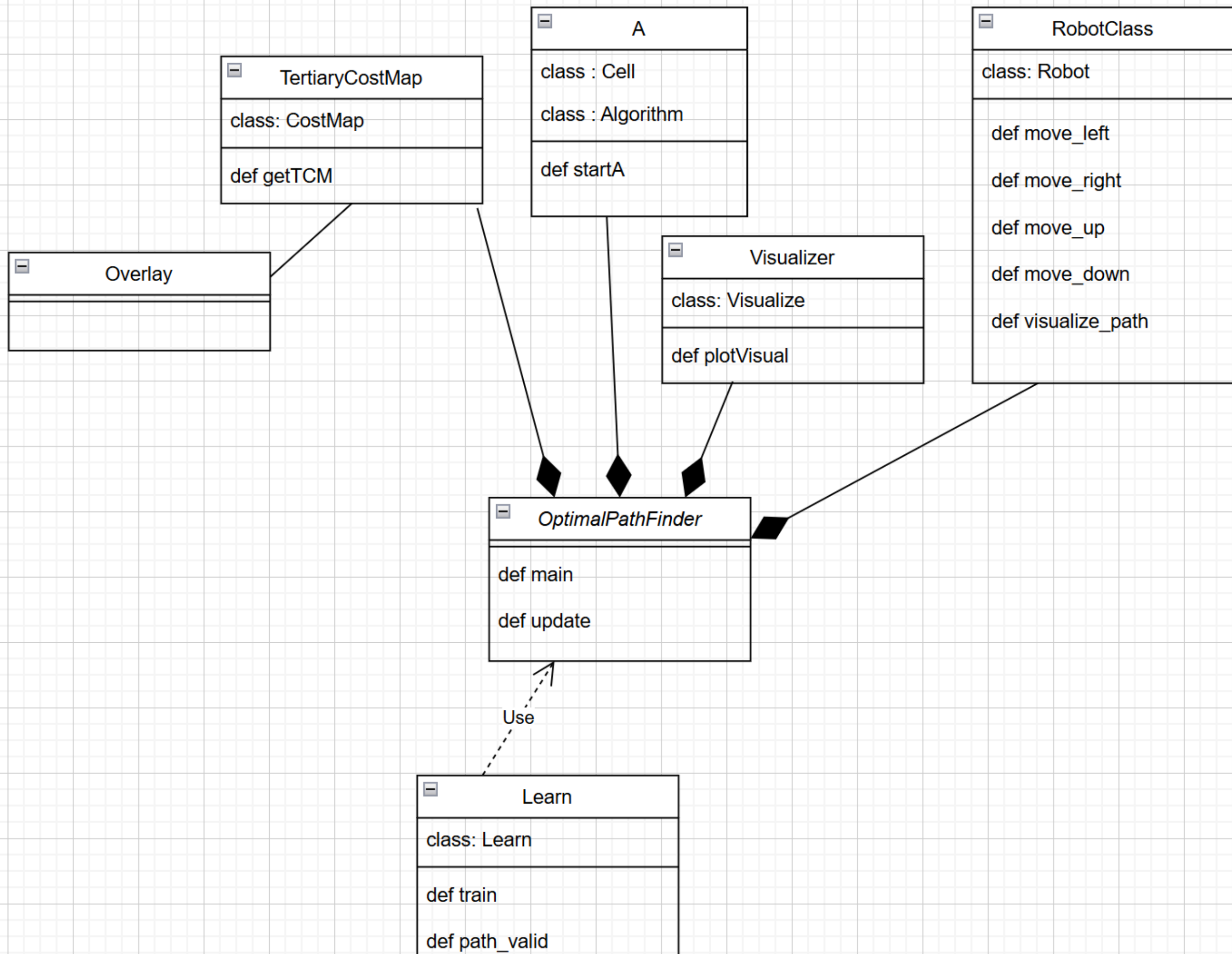
Step 5:

- Generated random coordinates within graph to become obstacles
 - Rand function and loop to assign indices to 0

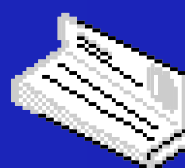
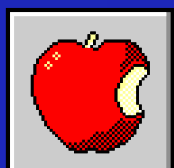
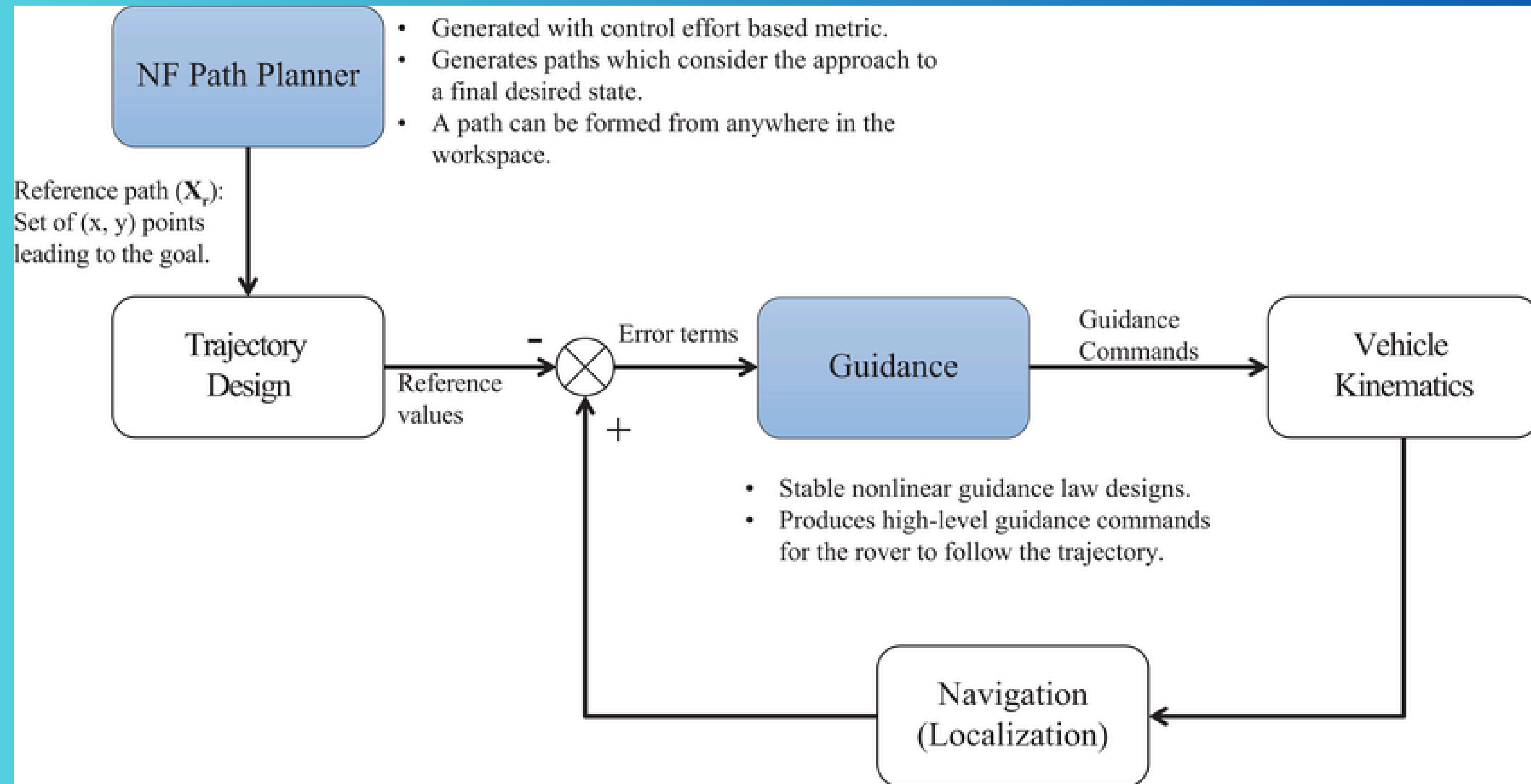
Step 6:

- Path coordinate values overrides any obstacles
 - Create a new path for moving around obstacle



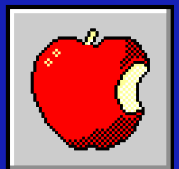


Design Choices and Rationale



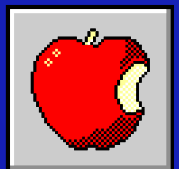
Team Collaboration and Role Distribution

- **Gavi:**
 - Creating images with distinct paths
 - Robot algorithm
- **George:**
 - Refactoring from pygame to matplotlib
 - Managing the environment
- **Jocelyn:**
 - Algorithm and logic/decision-making
 - Design/Structure



Packages and Libraries Used

- **Matplotlib**
 - Replacement for pygame
- **Rasterio**
 - Handling images and data
- **Numpy**
 - Creating arrays and calculation
- **GeoPandas**
 - Geological data handling
- **Other: math and heapq**



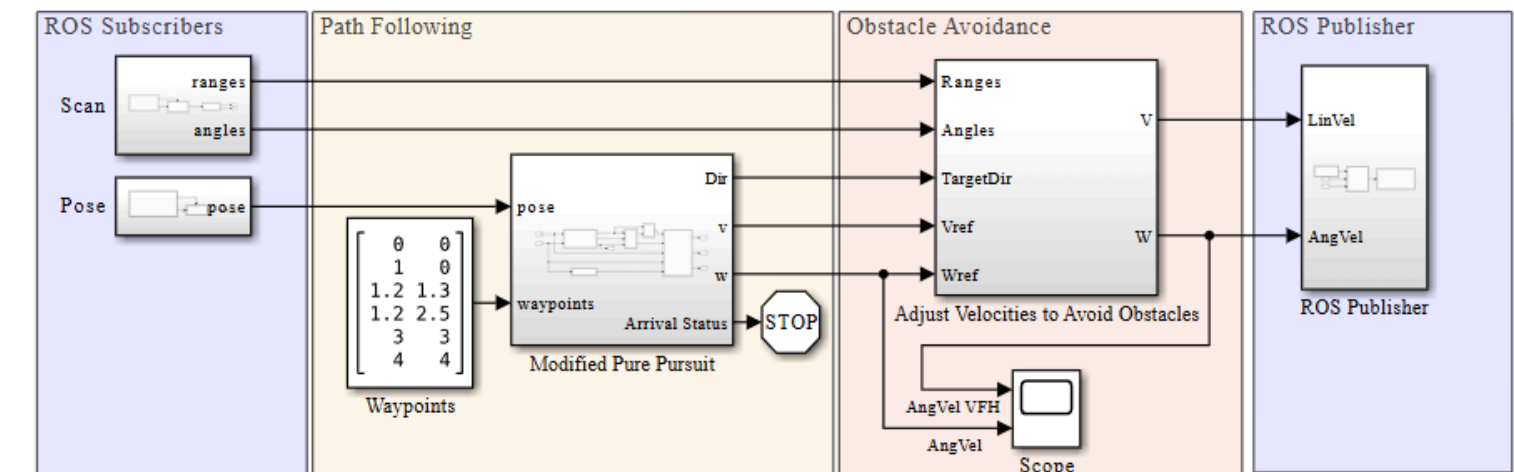
Innovative Approaches

Robot's algorithm:

- Researched different navigation and guidance models for mobile robots and unmanned ground vehicles (UGV) in Matlab and other platforms (such as A*)
- Researched ROS (Robot Operating System) enabled simulations for autonomous navigation of Turtlebot3 with obstacle avoidance in Matlab
 - createImageMask
 - convertMapToMatrix
 - LidarSlam
 - LidarSlam_interp
- This improves efficiency, reliability, and the ability to automate complex tasks for real-life implementation

Waypoint Following with Obstacle Avoidance + ROS Control

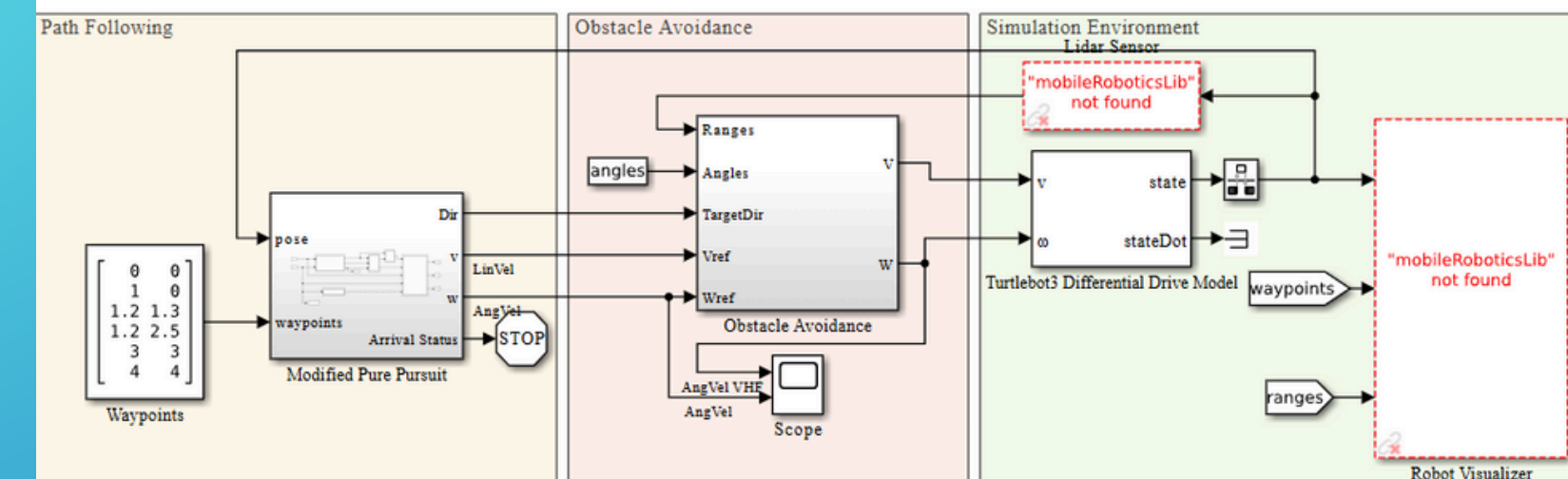
This Simulink® controls a ROS robot to perform path following with obstacle avoidance given a set of waypoints. The algorithm uses a [VFH \(Vector Field Histogram\)](#) block in order to identify collision-free behaviors by processing scan information from a lidar sensor.



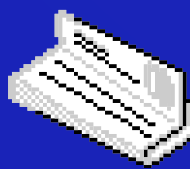
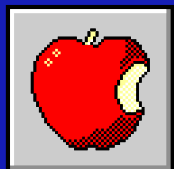
Copyright 2021 The MathWorks, Inc.

Waypoint Following with Obstacle Avoidance

This Simulink® model simulates path following with obstacle avoidance given a set of waypoints. The algorithm uses a [VFH \(Vector Field Histogram\)](#) block in order to identify collision-free behaviors by processing scan information from a lidar sensor.



Copyright 2021 The MathWorks, Inc.



[Back to Agenda Page](#)

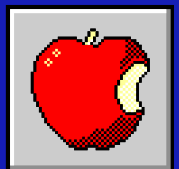
Innovative Approaches

Design choice utilizing matrices

Dedicating values within an array to hold significance, rather than creating multiple data structures.

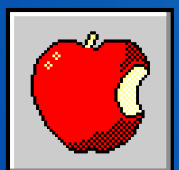
This:

- Saves time
- Saves allocation on the heap and stack

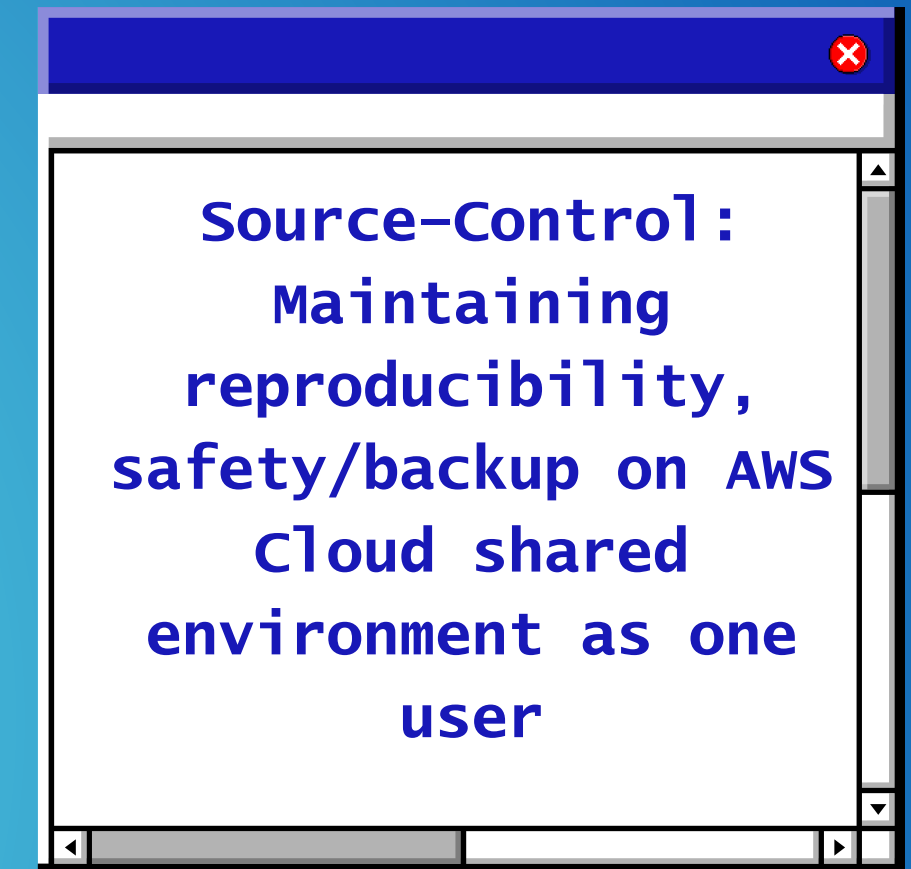
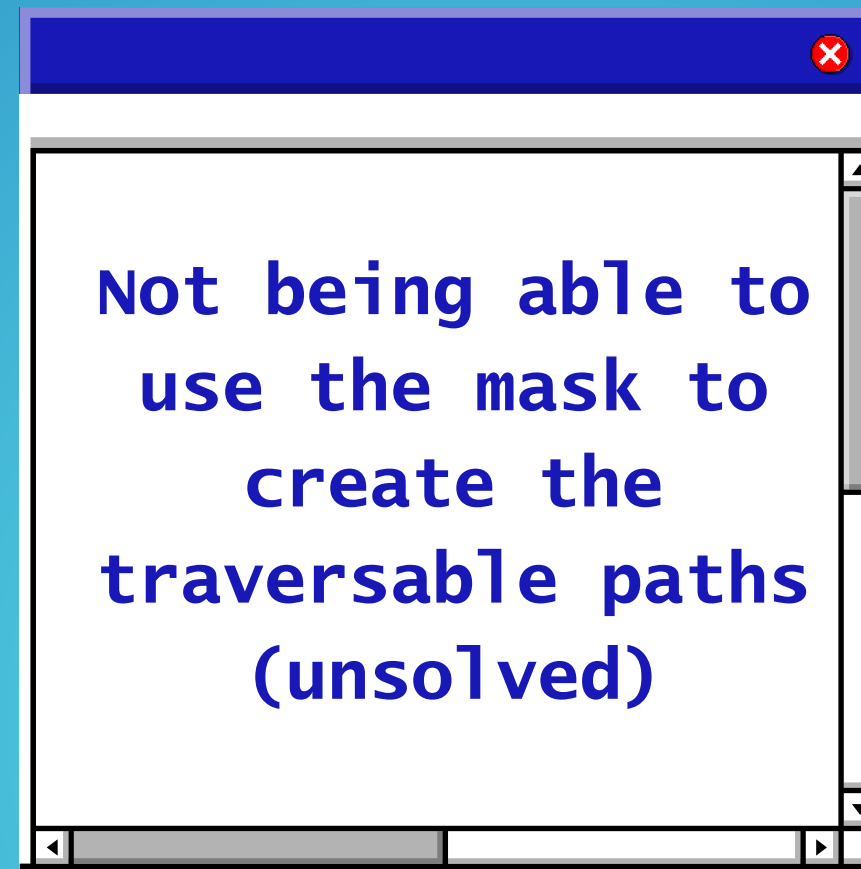
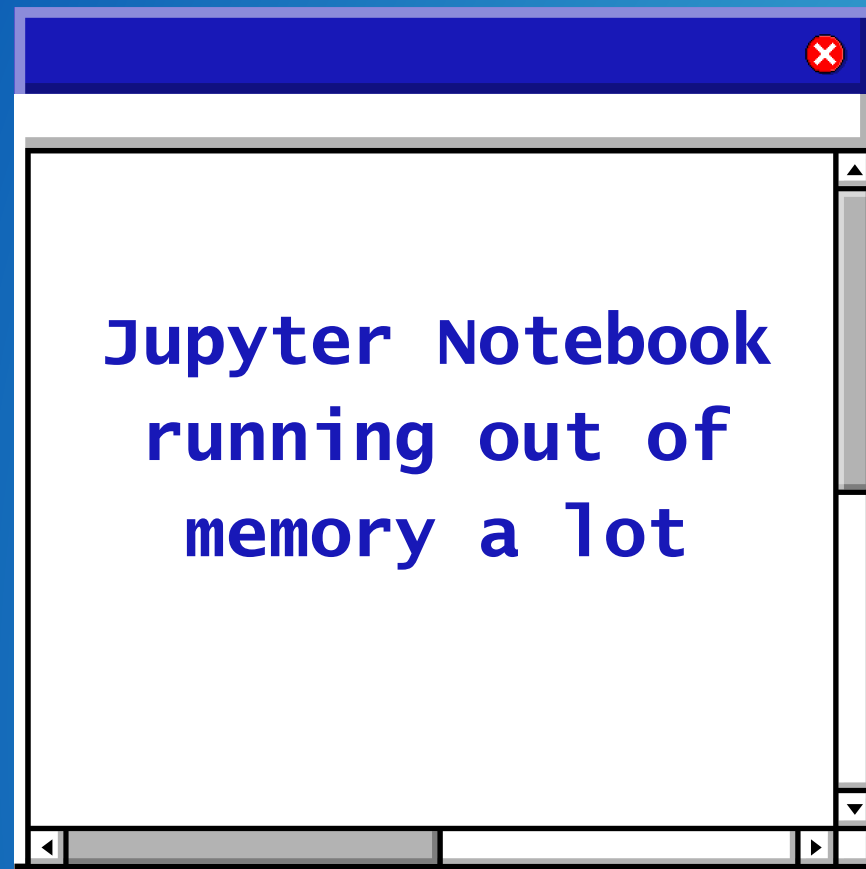


Factors Considered in the Model

- If there is an obstacle on the direct path, is there traversable path around we can use to avoid the object?
- What are the most accurate and time-efficient pathways that the robot can travel?
- What timesteps are needed for the robot to collect travel data?
- How to determine steep elevation changes and consider them as untraversable?
- How do those road boundary and DEM files provided work together to identify coordinates



Challenges Faced and Lessons Learned



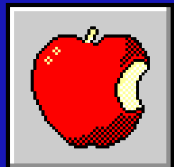
Lessons Learned

- Some solutions may come with drawbacks
- More applications to Linear Algebra

what we would Do Differently



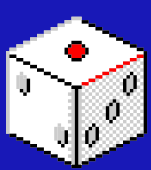
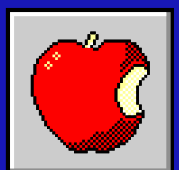
- Change to the GPU profile earlier
 - *Calculating and maintaining a prediction of time and problem complexity in order to anticipate these changes*
- AND Deciding to change direction early on if something isn't working (be less stubborn!)



Future Development Plans with Additional Time

Use documented robotic navigation schemes such as:

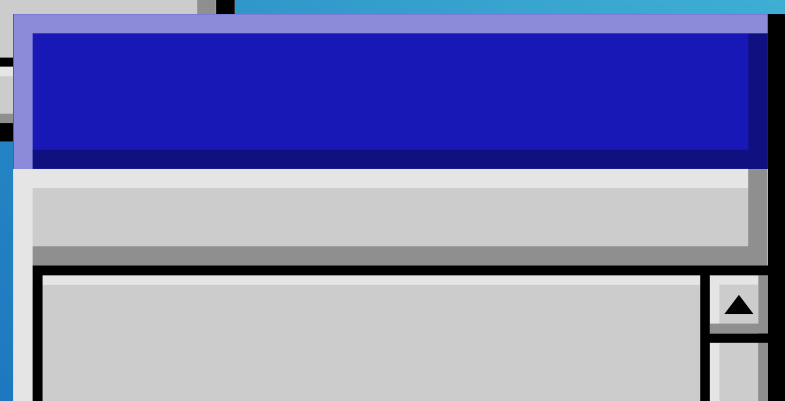
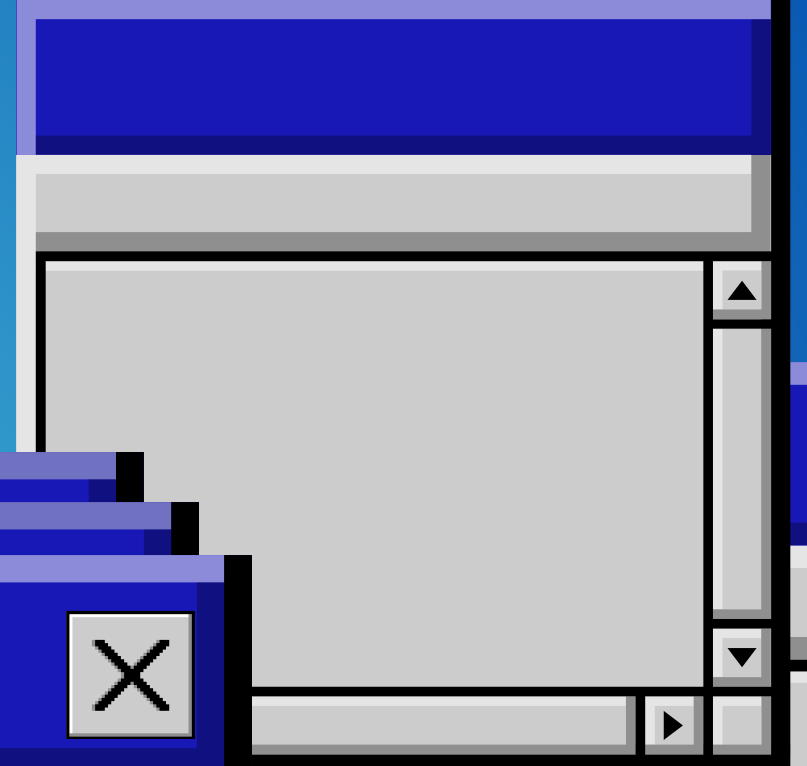
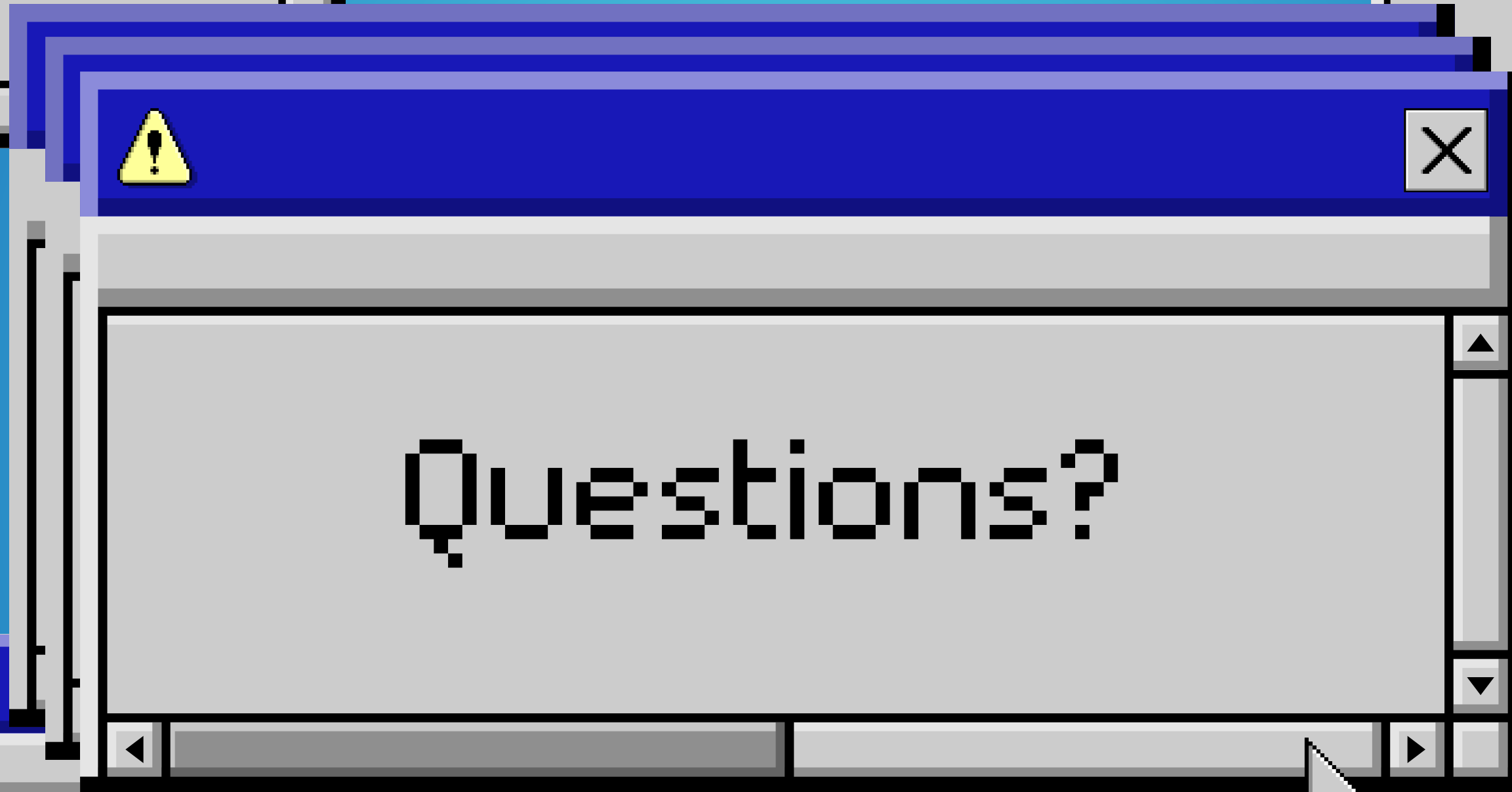
- Implement Monte Carlo Localization (MCL) with Optimized Path
 - Determines moving vehicle's position within an environment using a map, sensors, and a set of random samples (particles) to represent the robot's belief about state.
 - Helps in interpreting noisy sensor data, by generating multiple hypotheses about the world state and updating beliefs based on the likelihood of sensor readings.
- Focus more on iterating the model
 - Machine Learning training optimization





**Thank you to NASA, USDA, US
Forest Service, AWS, and CSU
for this amazing opportunity!**

[Back to Agenda Page](#)



works cited

- Matlab's Monte Carlo Localization Algorithm
- Image for GNC Flowchart
- Simulinks for Autonomous Navigation for Mobile Robots and UGV
- A Algorithm

